Faculty of Engineering and Technology Electrical and Computer Engineering Department

Digital Systems (ENCS2340)

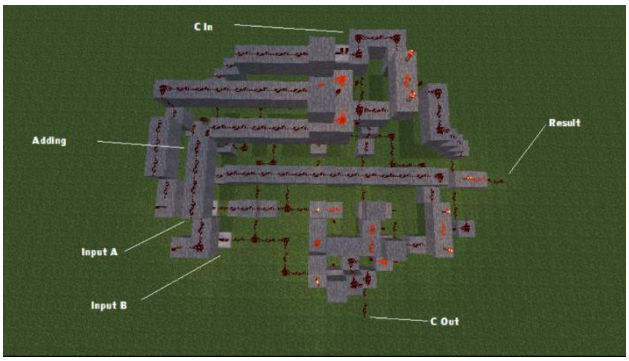
Verilog HDL Project

Student Name: *Shatha Saif eddien Abualrub*

student ID: 1231279

Section: 2

This report presents the design and implementation of a 4-bit Arithmetic Logic Unit (ALU) using Verilog HDL. The ALU performs addition, subtraction, bitwise AND, and bitwise OR operations. Overflow and carry-out detection mechanisms are integrated. The design was implemented using structural, dataflow, and behavioral modeling techniques. Simulation results confirm the correctness of the design.

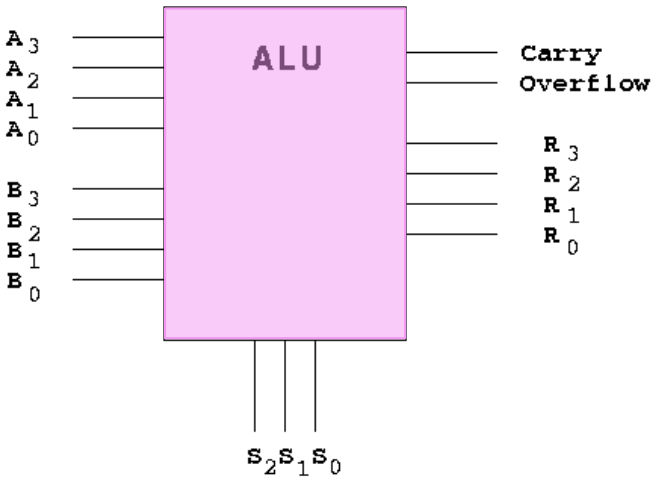


1- Introduction:

This project focuses on designing and building a simple **Arithmetic Logic Unit (ALU)** using Verilog HDL. An ALU is an essential part of digital systems, responsible for performing basic arithmetic and logical operations.

The ALU we designed can handle four main operations: addition, subtraction, bitwise AND, and bitwise OR. It takes two 4-bit inputs, a 3-bit control signal to choose the operation, and produces the result. Additionally, it outputs flags for carry-out (Cout) and overflow.

This project demonstrates the use of Verilog to model hardware and explores three key techniques for design: structural, dataflow, and behavioral modeling.



2- Design:

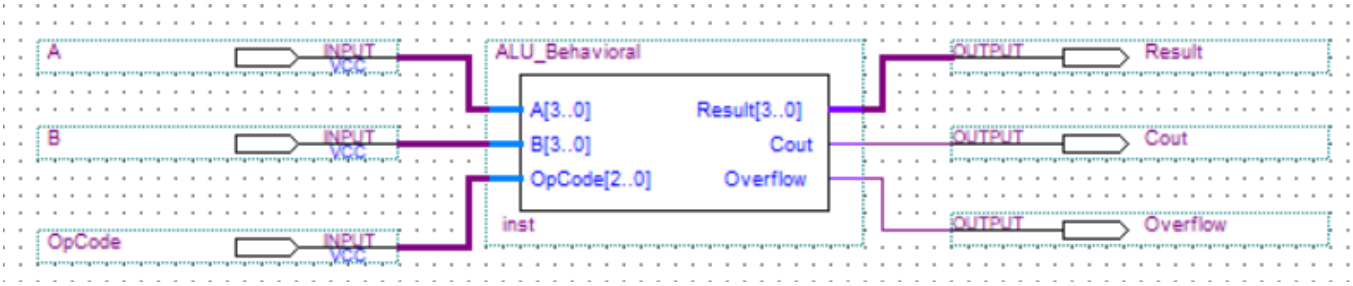
3.1 ALU Functionalities

- **Addition**: Computes the sum of two 4-bit operands.
- **Subtraction**: Performs 2's complement subtraction.
- **Bitwise AND**: Computes the logical AND of two 4-bit operands.
- **Bitwise OR**: Computes the logical OR of two 4-bit operands.

3.2 OpCode Mapping

OpCode	Operation
`3'b000`	Subtraction
`3'b001`	Addition
`3'b010`	Bitwise OR
`3'b011`	Bitwise AND

3.3 Block Diagram



3.4 Design Methodology

The ALU was designed using a modular approach. Core functionalities were implemented in separate modules:

- `Adder_4`: Performs addition with overflow detection.
- `Subtractor_4`: Handles 2's complement subtraction.
- `O_R`: Implements bitwise OR.
- `AND_`: Implements bitwise AND.

4- Verilog Code:

4.1 FullAdder Module

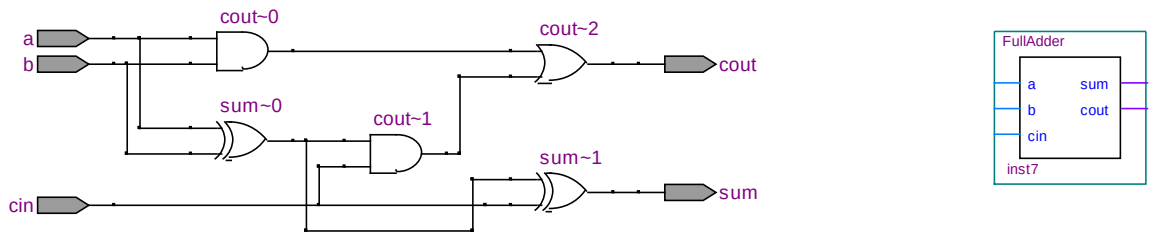
The FullAdder module is the basic building block for addition. It takes three inputs: two bits (A and B) and a carry-in (Cin), and produces two outputs: the sum (Sum) and the carry-out (Cout). This module is used to construct larger adders, like the 4-bit adder.

Code Implementation:

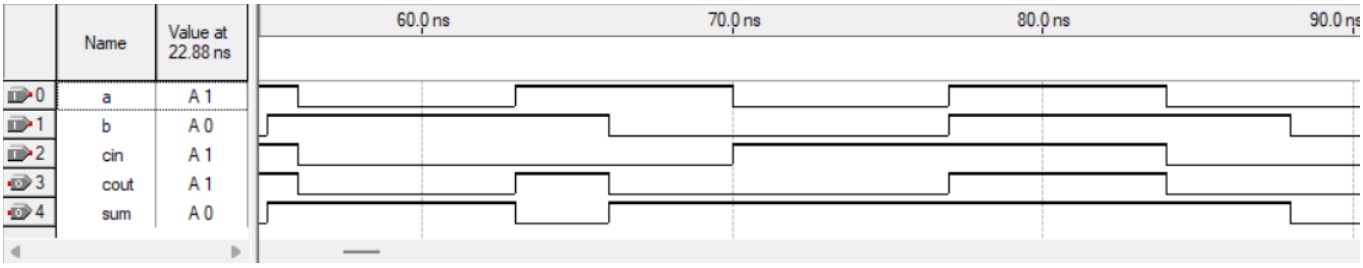
FullAdder.v

```
module FullAdder (input a , input b , input cin , output sum , output cout);
    assign sum = a ^ b ^ cin;
    assign cout = (a & b) | (cin & (a ^ b));
endmodule
```

Schematics/block diagram:



Simulation:



4.2 4-bit Adder Module

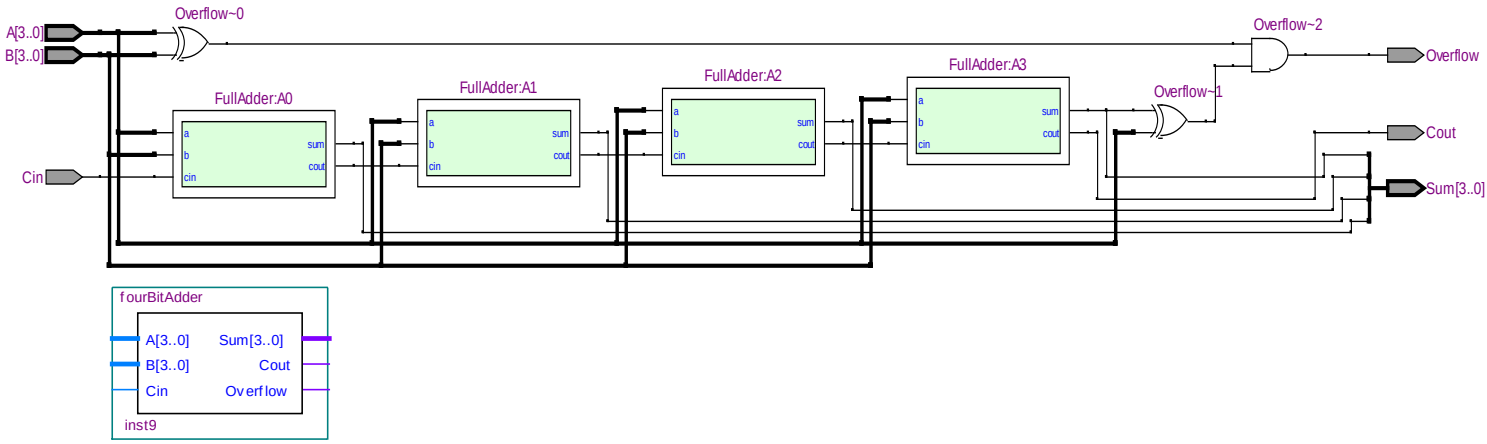
The fourBitAdder module performs 4-bit addition. It uses four FullAdder modules to add two 4-bit numbers (A and B) and produces a 4-bit result (Sum) along with a carry-out (Cout). It also detects overflow, which occurs when the result is too large to fit in 4 bits. Verilog Code:

Code Implementation:

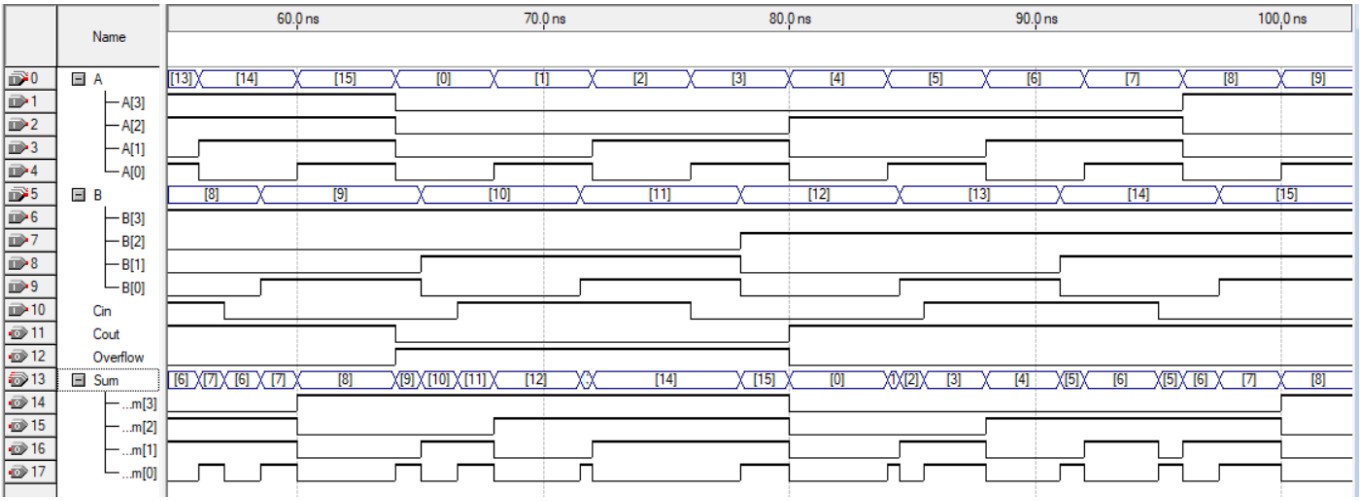
fourBitAdder.v

```
module fourBitAdder(input [3:0] A, B, input Cin, output [3:0] Sum, output Cout, output Overflow);
    wire [2:0] C;
    wire S_MSB;

    FullAdder A0 (A[0], B[0], Cin, Sum[0], C[0]);
    FullAdder A1 (A[1], B[1], C[0], Sum[1], C[1]);
    FullAdder A2 (A[2], B[2], C[1], Sum[2], C[2]);
    FullAdder A3 (A[3], B[3], C[2], Sum[3], Cout);

    assign S_MSB = Sum[3];
    assign Overflow = (A[3] ^ B[3]) & (S_MSB ^ A[3]); // Overflow detection
endmodule
```

Schematics:



Simulation:



4.3 4-bit Subtractor Module

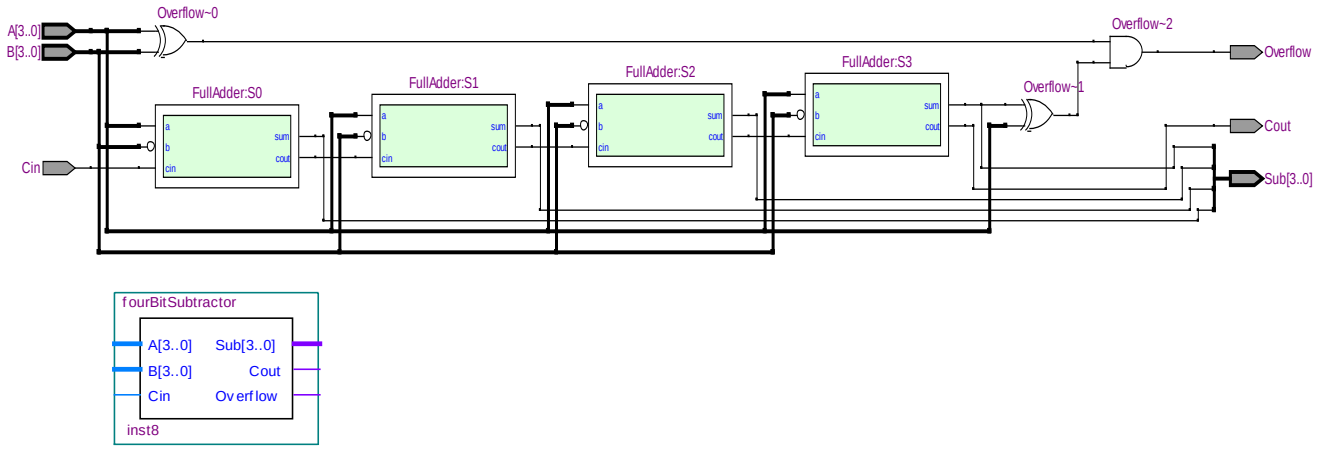
The fourBitSubtractor module handles 4-bit subtraction using 2's complement. It takes two 4-bit inputs (A and B) and produces a 4-bit result (Sub) along with a carry-out (Cout). It also detects overflow for subtraction, which occurs when the result is outside the representable range.

Code Implementation:

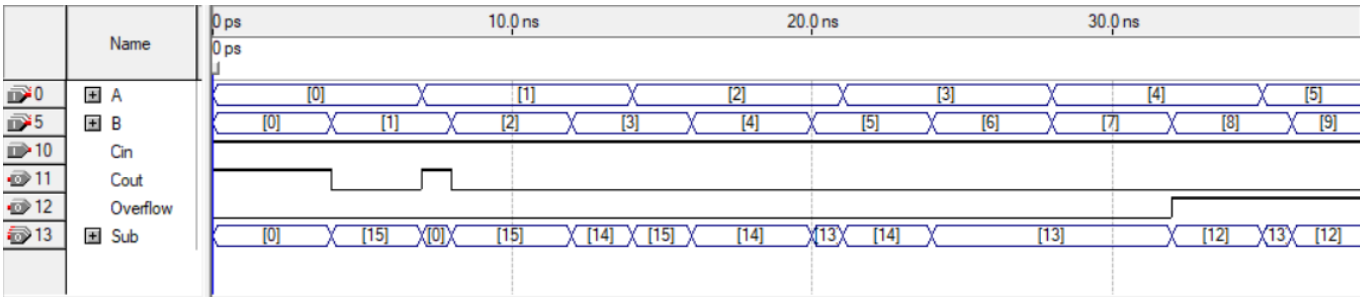
fourBitSubtractor.v

```
module fourBitSubtractor(input [3:0] A , B, input Cin, output [3:0] Sub, output Cout , Overflow);
    wire [3:0] B_1s;  // 1's complement of B
    wire [2:0] Carry;

    assign B_1s = ~B;

    FullAdder S0(A[0], B_1s[0], Cin, Sub[0], Carry[0]);
    FullAdder S1(A[1], B_1s[1], Carry[0], Sub[1], Carry[1]);
    FullAdder S2(A[2], B_1s[2], Carry[1], Sub[2], Carry[2]);
    FullAdder S3(A[3], B_1s[3], Carry[2], Sub[3], Cout);


    assign Overflow = (A[3] ^ B[3]) & (A[3] ^ Sub[3]);// Overflow detection
endmodule
```

Schematics/block diagram:



Simulation:



4.4 Bitwise-And Module

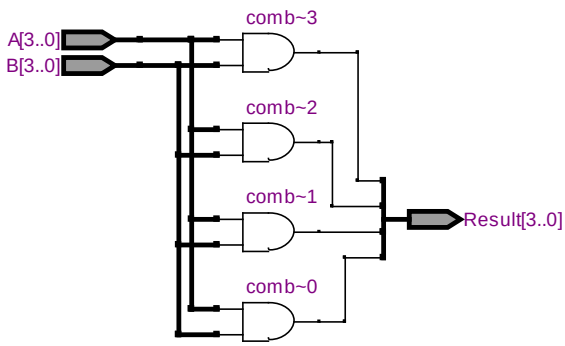
The BitWise_And module performs a bitwise AND operation on two 4-bit inputs (A and B). It compares each corresponding bit of A and B and produces a 4-bit result where each bit is the logical AND of the input bits.

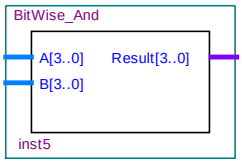
Code Implementation:

BitWise_And.v

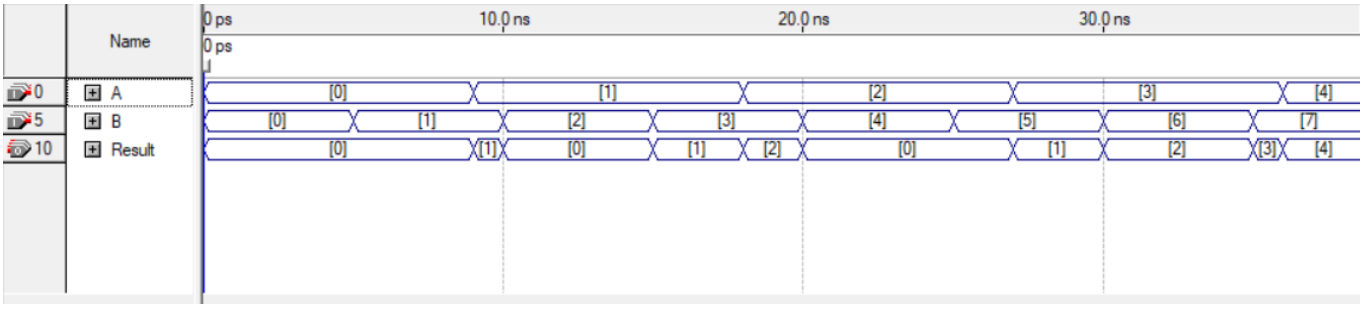
```
module BitWise_And(input [3:0] A,B,output [3:0] Result);

    and (Result[0], A[0], B[0]);
    and (Result[1], A[1], B[1]);
    and (Result[2], A[2], B[2]);
    and (Result[3], A[3], B[3]);

endmodule
```

Schematics/Block diagram:





Simulation:



4.5 Bitwise-Or Module

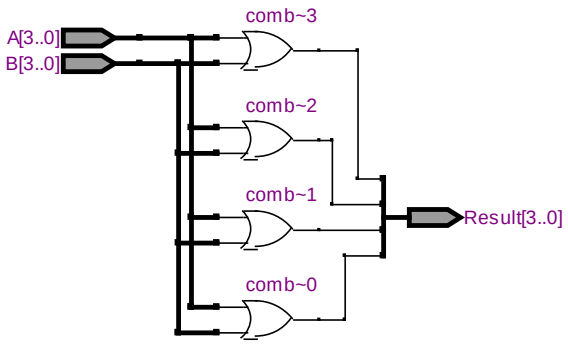
The BitWise_Or module performs a bitwise OR operation on two 4-bit inputs (A and B). It compares each corresponding bit of A and B and produces a 4-bit result where each bit is the logical OR of the input bits.

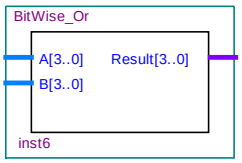
Code Implementation:

BitWise_Or.v

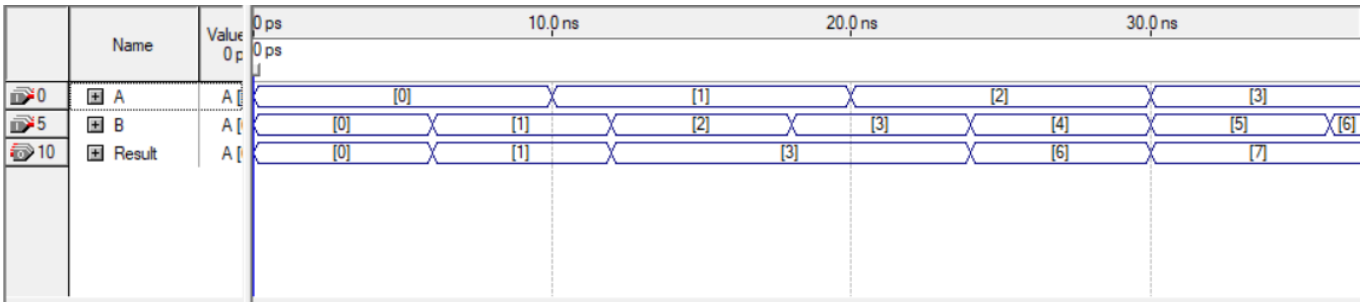
```
module BitWise_Or(input [3:0] A,B,output [3:0] Result);

    or (Result[0], A[0], B[0]);
    or (Result[1], A[1], B[1]);
    or (Result[2], A[2], B[2]);
    or (Result[3], A[3], B[3]);

endmodule
```

Schematics/block diagram:





Simulation:



4.6 ALU Module (Data Flow Modeling)

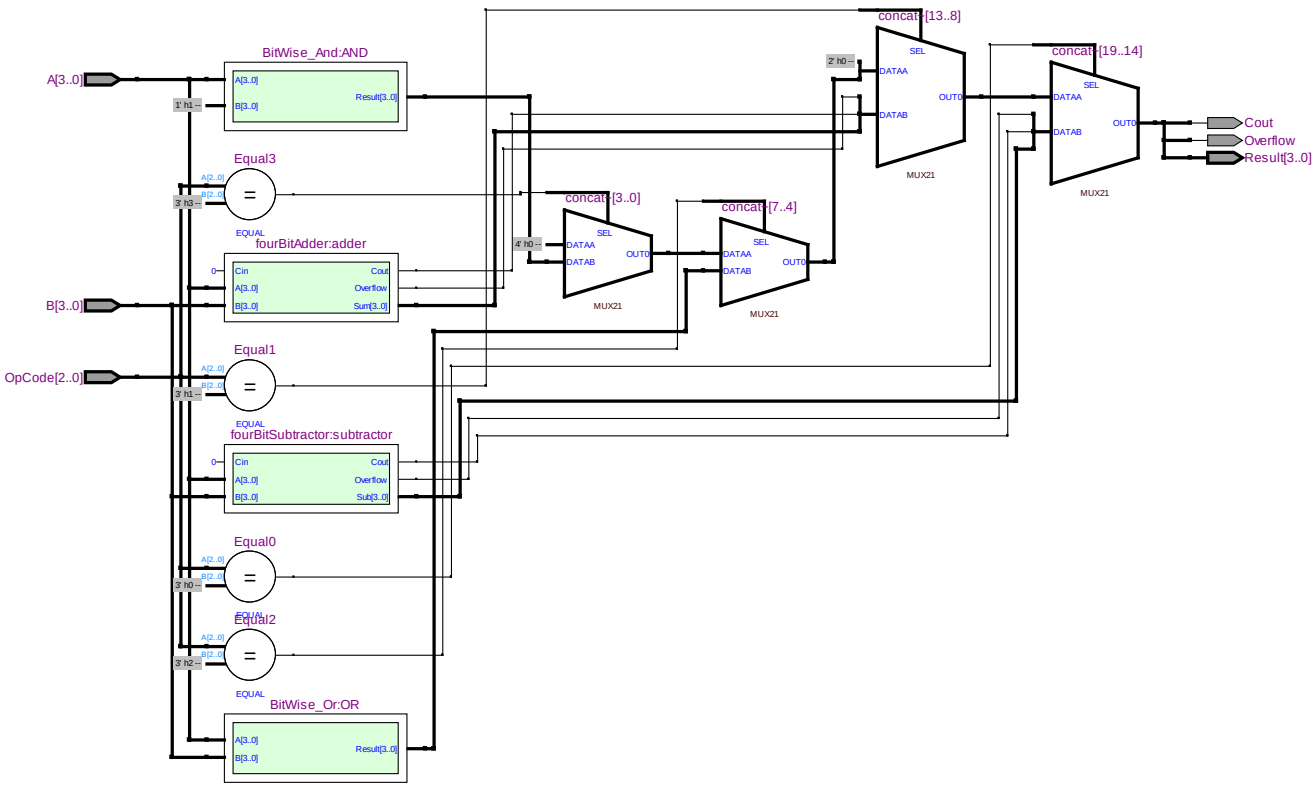
The ALU module is the top-level module that integrates all the operations. It takes two 4-bit inputs (A and B) and a 3-bit control signal (OpCode) to select the operation (addition, subtraction, AND, or OR). It uses a multiplexer to route the correct result to the output based on the OpCode. The module also outputs carry-out (Cout) and overflow (Overflow) for arithmetic operations.

Code Implementation:

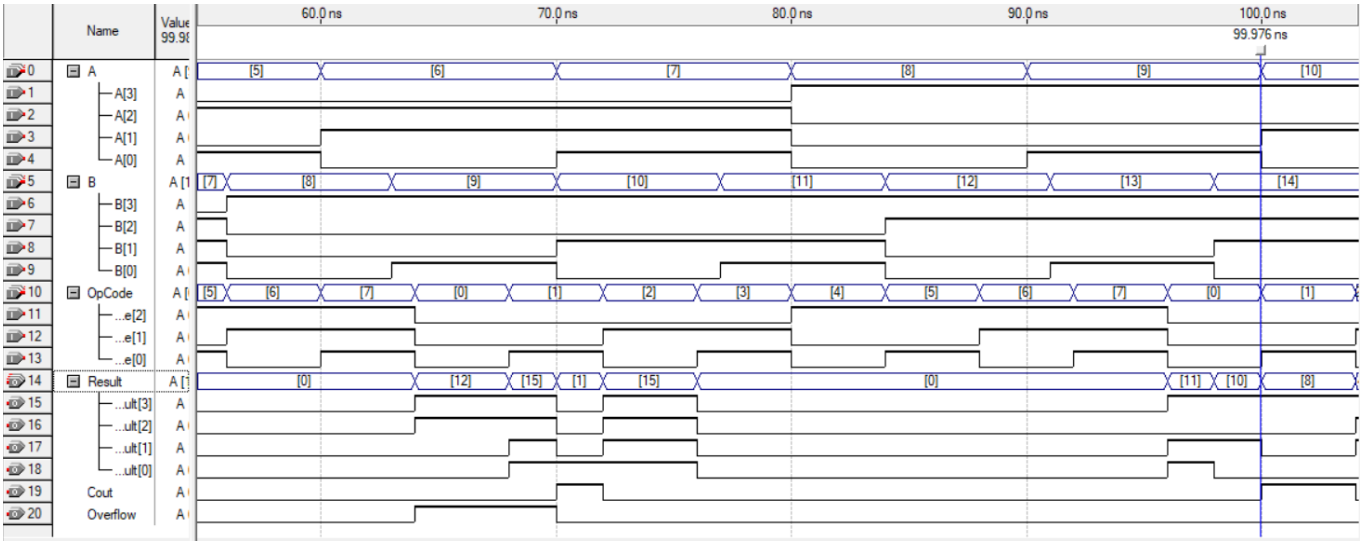
ALU_Dataflow.v*

```
module ALU_Dataflow(input [3:0] A , B , input [2:0] OpCode , output [3:0] Result , output Cout , Overflow);

    wire [3:0] sum, sub, or_result, and_result;
    wire cout_add, cout_sub, overflow_add, overflow_sub;

    fourBitAdder adder(A, B, 1'b0, sum, cout_add, overflow_add);
    fourBitSubtractor subtractor(A, B, 1'b0, sub, cout_sub, overflow_sub);
    BitWise_Or OR(A, B, or_result);
    BitWise_And AND(A, BS, and_result);

    assign {Result, Cout, Overflow} = (OpCode == 3'b000) ? {sub, cout_sub, overflow_sub} :
                                      (OpCode == 3'b001) ? {sum, cout_add, overflow_add} :
                                      (OpCode == 3'b010) ? {or_result, 1'b0, 1'b0} :
                                      (OpCode == 3'b011) ? {and_result, 1'b0, 1'b0} :
                                                           {4'b0000, 1'b0, 1'b0};
endmodule
```

Schematics:



Simulation:



4.7 ALU Module (Behavioral Modeling)

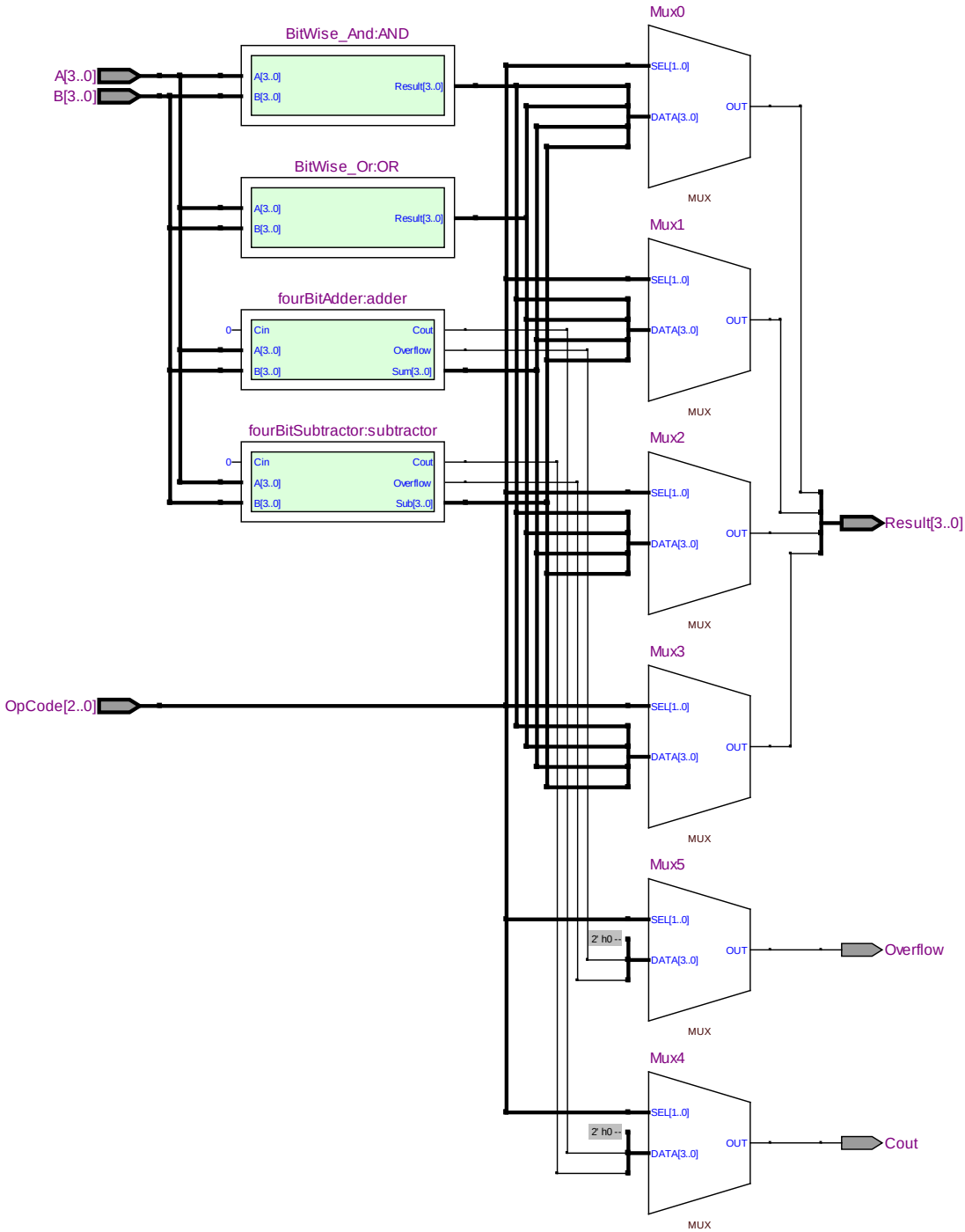
The ALU_Behavioral module implements the same functionality as the ALU module but uses behavioral modeling. It uses a case statement to select the operation based on the OpCode. This approach is more flexible and easier to modify for additional operations in the future.

Code Implementation:

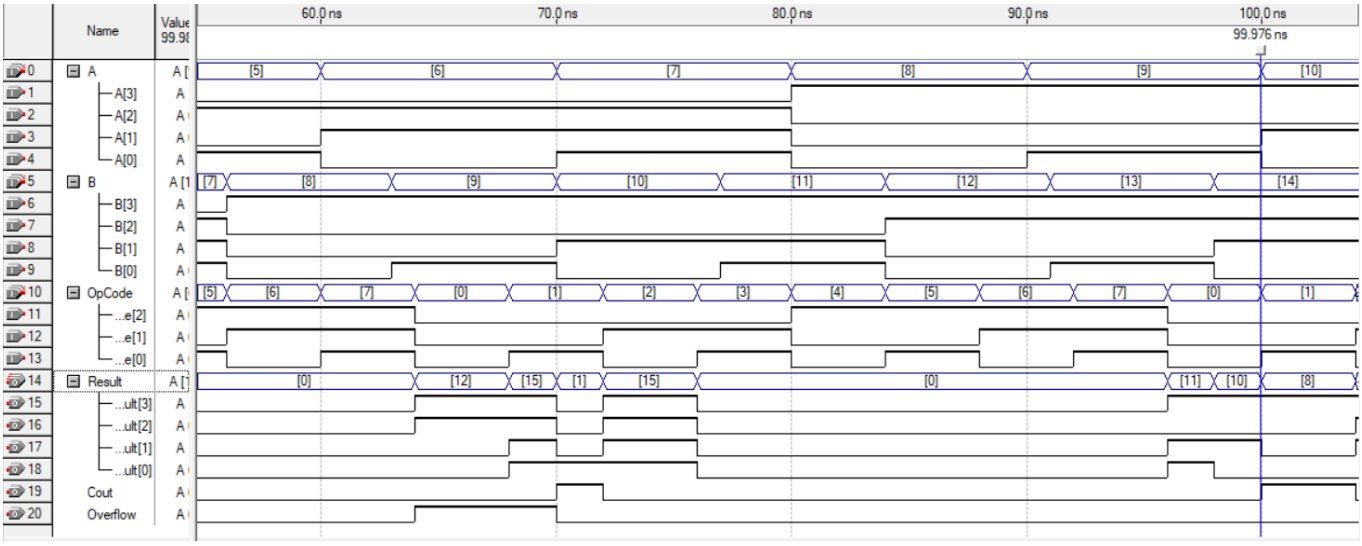
ALU_Behavioral.v

```
module ALU_Behavioral( input [3:0] A , B, input [2:0] OpCode , output reg [3:0] Result , output reg Cout , Overflow);
    wire [3:0] sum, sub, or_result, and_result;
    wire cout_add, cout_sub, overflow_add, overflow_sub;


    fourBitAdder adder(A, B, 1'b0, sum, cout_add, overflow_add);
    fourBitSubtractor subtractor(A, B, 1'b0, sub, cout_sub, overflow_sub);
    BitWise_Or OR(A, B, or_result);
    BitWise_And AND(A, B, and_result);

    always @(*) begin
        case (OpCode)
            3'b000: {Result, Cout, Overflow} = {sub, cout_sub, overflow_sub};
            3'b001: {Result, Cout, Overflow} = {sum, cout_add, overflow_add};
            3'b010: {Result, Cout, Overflow} = {or_result, 1'b0, 1'b0};
            3'b011: {Result, Cout, Overflow} = {and_result, 1'b0, 1'b0};
            default:{Result, Cout, Overflow} = {4'bxxxx, 1'bx, 1'bx};
        endcase
    end
endmodule
```

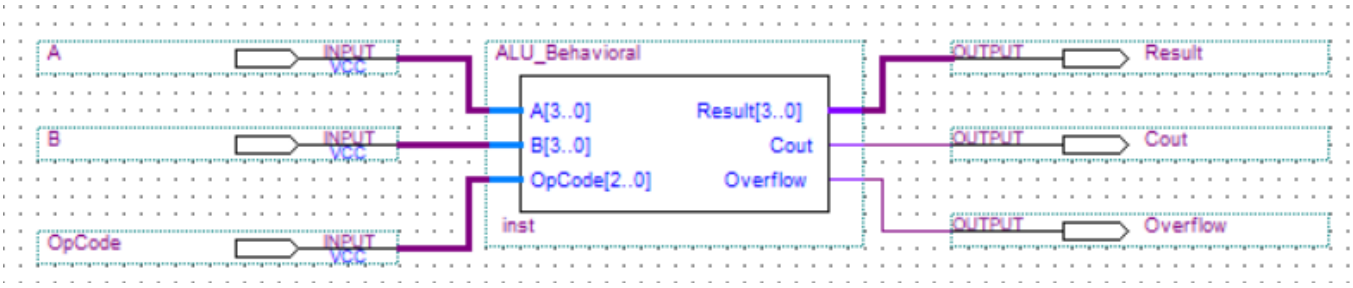
Schematics:


A[3..0]
B[3..0]
BitWise_And:AND
A[3..0]
B[3..0]
Result[3..0]
BitWise_Or:OR
A[3..0]
B[3..0]
Result[3..0]
fourBitAdder:adder
0
Cin
A[3..0]
B[3..0]
Cout
Overflow
Sum[3..0]
fourBitSubtractor:subtractor
0
Cin
A[3..0]
B[3..0]
Cout
Overflow
Sub[3..0]
OpCode[2..0]
Mux0
SEL[1..0]
DATA[3..0]
OUT
MUX
Mux1
SEL[1..0]
DATA[3..0]
OUT
MUX
Mux2
SEL[1..0]
DATA[3..0]
OUT
MUX
Mux3
SEL[1..0]
DATA[3..0]
OUT
MUX
Mux5
SEL[1..0]
2' h0
DATA[3..0]
OUT
MUX
Mux4
SEL[1..0]
2' h0
DATA[3..0]
OUT
MUX
Result[3..0]
Overflow
Cout

Simulation:



Block Diagram:



5- Conclusion:

This project designed and implemented a 4-bit ALU using Verilog HDL, capable of performing addition, subtraction, bitwise AND, and bitwise OR operations. The design utilized structural, dataflow, and behavioral modeling, with modular components for each operation. Simulation results confirmed the ALU's correctness, and the system successfully handled various input combinations. Future work could expand the ALU's functionality or increase its bit width. This project demonstrated the effective use of Verilog HDL for digital design.